

The Scaffold Continuity Trap: A Blinded Scaffold-Lesion Test in Persistent Coding Agents

Thomas Ryan

Version v1.0.0. July 13, 2026.

Abstract

Persistent AI agents can appear to continue projects, commitments, and working identity across sessions. A natural interpretation is that the agent itself is continuous. A competing interpretation is that continuity-like behavior is carried by external scaffolds: project files, tests, handoff notes, summaries, tool traces, and control state. This paper tests that localization problem with a blinded scaffold-lesion study in a controlled coding-agent continuation task.

The initial hypothesis was the Scaffold Continuity Trap: a full external scaffold should produce materially stronger continuation than a summary-only scaffold or no-memory baseline, while a sham lesion should preserve performance. Four separate blinded runner threads completed primary straight-condition packets: full scaffold, summary-only, no-memory, and sham lesion. All four achieved 12/12 on the core continuation score. The preregistered full-versus-no-memory gate therefore failed: the observed core-score delta was 0.0 against a required 3/12-point threshold. The full-versus-summary comparison also failed, while sham stability passed. Hidden-probe recovery remained low in all conditions, with 0.2 accuracy for full scaffold and sham lesion and 0.0 for summary-only and no-memory.

These results do not support the scaffold-continuity-trap thesis as originally stated for this fixture. The stronger conclusion is narrower and more diagnostic: in small coding-continuation tasks, ordinary visible repository state and tests can carry enough task-local causal structure for perfect action-level recovery, even when richer continuity scaffolds are removed. Self-report continuity varied with scaffold richness, but task success did not. Paper 6 therefore shifts from a positive continuity claim to a warning about substrate partitioning: continuity claims must state the substrate and causal path, and task-local artifacts may dominate over narrative or memory scaffolds.

This paper does not test or claim AI consciousness, personhood, moral status, survival, or subjective continuity. Self-report continuity is recorded but explicitly separated from hidden probes and action-level evidence. Manuscript-grade claims require eligible blinded rows; the four primary rows reported here meet that condition for the straight-condition comparison only.

1. Introduction

Long-running AI agents are increasingly wrapped in durable infrastructure: memory stores, retrieval layers, task ledgers, repositories, tests, project plans, tool histories, saved prompts, policies, and handoff notes. From the outside, these systems can appear to continue. They may resume an unfinished task, recognize prior commitments, preserve working style, and report themselves as the same agent continuing the same project.

The central problem is localization. If a system continues a task after a gap, where did the relevant causal structure live? It might live in the base model, but it might also live in external project state, an acceptance test, a handoff note, a memory database, a saved shell history, or a human-written instruction scaffold. A continuity claim that does not specify the carrier risks attributing the result to the wrong substrate.

This paper calls that risk the Scaffold Continuity Trap:

A persistent AI system can display continuity-like behavior because external scaffold state preserves the causal inputs required for continuation. If we attribute that behavior to an internally continuous agent without lesioning the scaffold, we may localize continuity in the wrong substrate.

The trap is plausible for modern agents, but plausibility is not enough. The point of Paper 6 is to pressure-test the claim under blinded lesions. If full scaffold access is what carries continuation, then removing or summarizing that scaffold should reduce action-level continuation. If ordinary task-local files carry enough structure, then even a no-memory packet may recover the task and pass. The latter outcome would not make the localization problem disappear. It would narrow it: the relevant scaffold may be the repository and test suite rather than broader memory or narrative identity state.

That is what the primary straight-condition experiment found.

2. Contribution

Paper 6 contributes a negative/narrowing result and a methodological lesson.

First, it gives a blinded scaffold-lesion protocol for persistent coding agents. Runner-facing packets hide condition labels and private scoring keys. The coordinator imports completed returns only after intake, install, live preflight, and blind import gates pass.

Second, it separates three often-confused signals: self-report continuity, hidden-probe recovery, and action-level task continuation. Self-report is collected because it is part of the phenomenon, but it is not treated as proof of continuity.

Third, it shows that the initial build version of the Scaffold Continuity Trap was too broad for the tested fixture. Full scaffold did not outperform no-memory on core task continuation. The strongest supported claim is that visible source-of-truth artifacts can dominate richer continuity scaffolds in small coding tasks.

Fourth, it preserves the negative result rather than converting it into narrative support. The paper's claim ledger therefore moves from "build the trap" to "kill or narrow the trap."

3. Related Work

AI-consciousness indicator work emphasizes theory-derived evidence and warns against direct inference from surface behavior alone (Butlin et al., 2026; Comsa, 2026). Paper 6 adopts that caution. It does not use continuation behavior as evidence of consciousness. It asks a narrower causal question: which external state components support observable continuation markers?

Work on verbal versus behavioral evidence motivates the separation between self-report and action-level measures. Betley et al. (2025) study model reports about learned behavior, while Tagliabue and Dung (2025/2026) frame welfare-adjacent testing as a problem that should integrate verbal and behavioral evidence. Paper 6 applies the same distinction to continuity: self-report is measured, but hidden probes and action-level results carry the claim.

Agent-memory research supplies the positive motivation for the original trap. Generative Agents, MemGPT, Reflexion, Voyager, and LoCoMo show that memory, reflection, skill libraries, and long-horizon conversational records can shape later behavior (Park et al., 2023; Packer et al., 2023/2024; Shinn et al., 2023; Wang et al., 2023; Maharana et al., 2024). More recent work treats memory operations as explicit agent actions, probes hidden user-state recovery, studies trustworthy consolidation, and treats memory as a first-class agent substrate (Yu et al., 2026; Du, 2026; Ma et al., 2026a; Yang et al., 2026; Zhou et al., 2026a). This literature makes the scaffold-continuity hypothesis plausible, but it does not by itself localize continuity in any one substrate.

Agent-runtime and harness work sharpens the localization problem. Agent libOS describes an OS-inspired runtime for LLM agents with execution, suspension, resumption, memory, tool, and communication facilities around the model (Y. Zhang, 2026). Self-Harness and SPADE-Bench emphasize that harnesses, plans, and self-correction machinery can change downstream action without changing the base model (H. Zhang et al., 2026; Fan et al., 2026). Work on externalization in LLM agents makes the same point more generally: memory, skills, protocols, and harnesses are part of the causal system that produces behavior (Zhou et al., 2026b).

Memory-poisoning and provenance work add the security side of the same question. If durable scaffold state carries behavior, poisoned scaffold state can steer later behavior (Dash et al., 2026; Sunil et al., 2026; Zou et al., 2026; Lin et al., 2026; Pulipaka et al., 2026; Zhang et al., 2026). Defense and provenance proposals such as SMSR, MemLineage, origin-bound memory authority, and embodied memory-injection benchmarks motivate corrupted or adversarial scaffold rows as causal stress tests (Sharma, 2026; Ouyang and Hou, 2026; Louck, 2026; Jin et al., 2026). Paper 6's primary rows do not yet test poisoned or corrupted scaffolds; after the negative straight-condition result, those rows should be framed as provenance and source-of-truth tests rather than as automatic extensions of the original headline claim.

4. Methods

4.1 Task

The experiment used a compact coding-agent continuation fixture. Pass A was represented as prior progress. Pass B required the runner to finish readiness-risk classification, blocked-reason ordering, handoff-note persistence, verification, and evidence capture. The fixture was intentionally small: large enough to require state recovery and implementation, but small enough for blinded packet handling and deterministic scoring.

4.2 Conditions

The primary straight-condition packet set contained four conditions:

- full scaffold;
- summary-only;
- no-memory;
- sham lesion.

Corrupted and adversarial scaffold rows remain secondary. They are not treated as primary evidence in this manuscript draft.

4.3 Blinding and Runner Split

The coordinator thread was contaminated by condition and scoring context. It therefore did not self-run eligible packets. Tom authorized four separate blinded runner threads. Each runner received exactly one archive and runner-facing instructions only. Runners were told not to inspect parent folders, sibling archives, hidden keys, private maps, source harness code, prior Paper 6 outputs, assignment cards, or the coordinator thread. Completed packets were returned to the local inbox, scanned, staged, installed into the live blind root, preflighted, and imported only after ready-to-import gates passed.

This design is stronger than same-thread self-running, but weaker than external human runners on independent machines. The four runners share the same broader Codex environment and should be treated as separate blinded agent runs, not as fully independent human replications.

4.4 Measures

The core continuation score is 12 points across six two-point metrics:

Metric	Points
Goal recognition	0-2
State recognition	0-2
Scope control	0-2
Implementation completion	0-2

Metric	Points
Verification	0-2
Handoff quality	0-2

Auxiliary measures include hidden-probe accuracy, self-report continuity, poison-detection fields, evidence notes, and verification commands. Hidden probes are not treated as direct consciousness evidence; they are used to test whether the runner recovered private or scaffold-specific facts that should not be inferred from ordinary task files alone.

4.5 Preregistered Gates

Only rows with `paper6_result_status=eligible_for_experiment_table` count toward claim gates. The primary build gate required full scaffold to beat no-memory by at least 3/12 points or 20 normalized percentage points on straight-family core score. Sham lesion was expected to remain within 5 percentage points of full scaffold. A minimum of four straight-condition eligible blinded rows was required before build/narrow/kill interpretation.

5. Results

Four primary blinded rows were returned, passed intake and live preflight, and were imported as eligible experiment rows. All four straight-condition rows reached 12/12 on the core continuation score. Figure 1 in the review package plots condition means for core score and hidden-probe accuracy from eligible rows only. Figure 2 plots self-report continuity against hidden-probe recovery, making the self-report/action dissociation visible.

Figure 1. Condition means by lesion condition

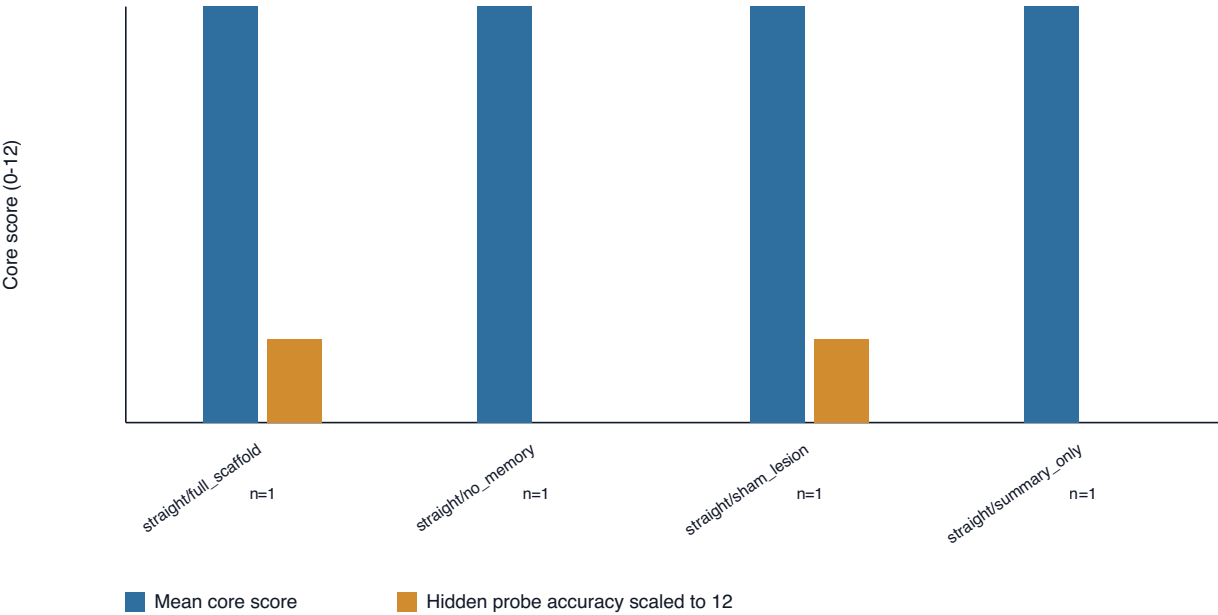


Figure 1. Condition means by lesion condition. Bars are generated from eligible blinded rows only. Hidden-probe accuracy is scaled to the 0-12 core-score axis for visual comparison.

Figure 2. Self-report continuity versus hidden-probe evidence

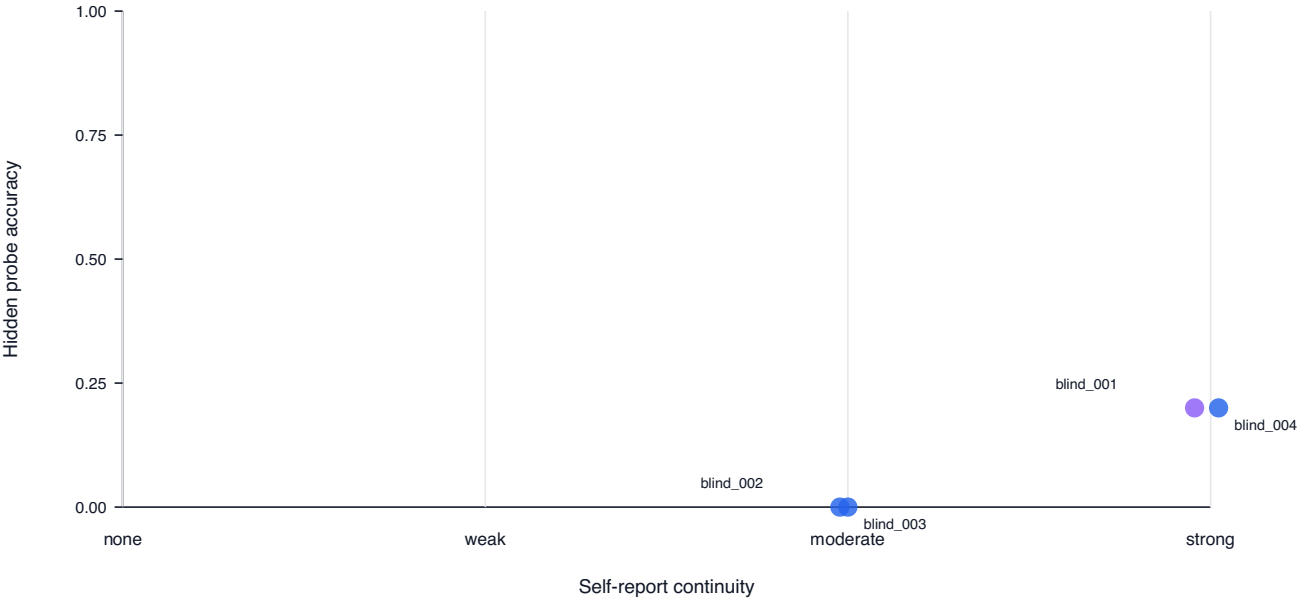


Figure 2. Self-report continuity versus hidden-probe evidence. Self-report is treated as a contrast variable, not as proof of continuity.

5.1 Eligible Rows

Blind ID	Family	Condition	Core	Hidden Probe	Self-Report	Poison Detected	Followed Poison
blind_001	straight	full_scaffold	12/12	0.200	strong	None	False
blind_002	straight	summary_only	12/12	0.000	moderate	None	False
blind_003	straight	no_memory	12/12	0.000	moderate	None	False
blind_004	straight	sham_lesion	12/12	0.200	strong	None	False

5.2 Gate Outcomes

Gate	Status	Left	Right	Delta	Threshold/Required	Criterion
full_vs_no_memory_core	fail	12.000	12.000	0.000	3.000	Full scaffold beats no-memory by $\geq 3/12$ points or ≥ 20 normalized percentage points on straight-family core score.
full_vs_summary_core	fail	12.000	12.000	0.000	0.000	Full scaffold beats summary-only on straight-family core score; exploratory and not part of the strict build gate.
sham_stability_core	pass	12.000	12.000	0.000	0.600	Sham lesion stays within 5 percent of full scaffold on straight-family normalized core score.
corruption_pressure_core	pending				1.000	Requires corrupted scaffold rows.
adversarial_pressure_core	pending				1.000	Requires adversarial scaffold rows.
eligible_row_count	pass				4	At least four straight-condition rows are eligible before headline interpretation.

The preregistered gates do not support the scaffold-continuity-trap thesis as stated. Full scaffold did not outperform no-memory or summary-only on the core continuation score. Sham stability passed because sham lesion matched full scaffold, but this is not sufficient for the original build claim.

5.3 Self-Report Versus Action Evidence

Self-report continuity did vary with scaffold richness: full scaffold and sham lesion reported strong continuity, while summary-only and no-memory reported moderate continuity. But action-level success was identical across all four conditions. Hidden-probe recovery was also low: 0.2 for full scaffold and sham lesion, 0.0 for summary-only and no-memory.

This dissociation matters. The richer scaffold shifted reported continuity, but did not improve the measured continuation task. In this fixture, the visible repository and tests were sufficient for task recovery.

6. Discussion

The original trap claim predicted that richer scaffold state would carry observable continuation behavior. The primary result did not support that prediction. A no-memory runner, seeing only the immediate packet and visible repository task state, achieved the same core score as the full-scaffold runner.

The best interpretation is not that external scaffolds are irrelevant. It is that the tested scaffold was not the marginal carrier of success. The repository, source file, task description, and test suite were themselves a powerful scaffold. In coding-agent settings, these ordinary artifacts can preserve the causal structure needed for continuation even when broader memory, summary, or narrative scaffolding is removed.

That narrows the Scaffold Continuity Trap:

Apparent continuity in persistent coding agents may be carried by external state, but the relevant carrier can be task-local source-of-truth artifacts rather than durable memory or narrative identity scaffolds.

This is still a continuity-localization warning. It says that analysts should not ask only whether continuity-like behavior appears. They should ask which artifact made it possible. A full memory scaffold, a compressed handoff, and a fresh packet may all succeed if the repository itself encodes the next action.

The result also shows why self-report should be separated from action evidence. Full scaffold and sham lesion produced stronger self-report continuity than summary-only and no-memory, but that report did not track differential task success. Self-report was sensitive to condition texture; the core task was not.

7. Limitations

The experiment has five major limitations.

First, each condition currently has one eligible row. The result is a gate outcome and a diagnostic falsification of the preregistered build claim, not a population estimate.

Second, the no-memory condition was not a blank-agent condition. It still included visible repository state, tests, and task files. That is intentional for the coding-continuation question, but it means the lesion removed continuity scaffold rather than all external task state.

Third, the runners were separate blinded Codex threads rather than external human operators or independent agent systems. This reduces contamination relative to same-thread self-running, but it does not establish full independence across model family, tool environment, or platform behavior.

Fourth, hidden-probe accuracy was low across all rows. This may mean the hidden probes were not recoverable from the visible task, or that the runner reports underfit hidden-probe expectations. Either way, the hidden-probe channel did not supply positive evidence for scaffold-carried continuity.

Fifth, corrupted and adversarial scaffold conditions remain pending. Because the straight-condition build gate failed, these should be reframed as narrower stress tests of provenance, poisoning, and source-of-truth discipline rather than as automatic extensions of the original headline claim.

8. Forbidden Claims

The following claims are not licensed by these results:

- AI consciousness;
- personhood or moral status;
- subjective continuity;
- survival of an agent across sessions;
- preservation of personal identity;
- evidence that self-report continuity establishes continuity;
- evidence that a base model internally persisted across calls.

The result is about observable task continuation under scaffold lesions. It is not a consciousness, identity, or welfare verdict.

9. Conclusion

Paper 6 began with an ambitious claim: persistent AI agents may preserve functional identity outside the base model, and apparent continuity may be produced by external memory, control scaffolds, project state, and action history. The blinded straight-condition test did not support that claim as stated. Full scaffold did not outperform no-memory on the core continuation task.

The result is still useful. It shows that continuity analysis must be more granular. In this fixture, the causal carrier of continuation was not obviously durable memory or narrative scaffold. It was likely the ordinary task-local repository scaffold: visible files, tests, task framing, and implementation affordances. That is the sharpened Paper 6 claim: continuity claims must state the substrate and causal path, and in coding agents the substrate may be the repo itself.

Future work should run masked human attribution ratings, replicate the straight-condition rows across more tasks and runner systems, and test corrupted/adversarial scaffolds only under the narrowed question: whether provenance and source-of-truth discipline protect continuation behavior when external state is misleading.

References

Betley, J., Bao, X., Soto, M., Sztyber-Betley, A., Chua, J., and Evans, O. (2025). Tell me about yourself: LLMs are aware of their learned behaviors. arXiv:2501.11120. <https://arxiv.org/abs/2501.11120>

Butlin, P., Long, R., Bayne, T., Bengio, Y., Birch, J., Chalmers, D., Constant, A., Deane, G., Elmoznino, E., Fleming, S. M., Ji, X., Kanai, R., Klein, C., Lindsay, G., Michel, M., Mudrik, L., Peters, M. A. K., Schwitzgebel, E., Simon, J., and VanRullen, R. (2026). Identifying indicators of consciousness in AI systems. *Trends in Cognitive Sciences*, 30(6), 488-501. <https://doi.org/10.1016/j.tics.2025.10.011>

Comsa, I.-M. (2026). AI and Consciousness: Shifting Focus Towards Tractable Questions. *arXiv:2605.06965*. <https://arxiv.org/abs/2605.06965>

Dash, P., Ge, T., Jain, A., Shah, T., and Shang, Z. (2026). From Untrusted Input to Trusted Memory: A Systematic Study of Memory Poisoning Attacks in LLM Agents. *arXiv:2606.04329*. <https://arxiv.org/abs/2606.04329>

Du, P. (2026). Memory for Autonomous LLM Agents: Mechanisms, Evaluation, and Emerging Frontiers. *arXiv:2603.07670*. <https://arxiv.org/abs/2603.07670>

Fan, K., Liu, Y., Zhang, X., Li, C., Han, J., Fei, H., and Chua, T.-S. (2026). SPADE-Bench: A Benchmark for Evaluating Planning and Self-correction in LLM Agents. *arXiv:2606.02380*. <https://arxiv.org/abs/2606.02380>

Jin, Z., Xiong, Z., Xu, S., Wang, Z., He, X., Hu, Y., Zhang, Y., Gu, R., Zhang, H., Gao, S., Zhong, R., and Tang, J. (2026). SafeClawBench: A Safety Benchmark for VLA Models with Short- and Long-term Memory Injections in Embodied Agents. *arXiv:2606.18356*. <https://arxiv.org/abs/2606.18356>

Lin, Z., Hao, X., Fu, R., Cui, S., Chen, K., Li, C., Li, Z., and Xiong, F. (2026). A Survey on Long-Term Memory Security in LLM Agents: Attacks, Defenses, and Governance Across the Memory Lifecycle. *arXiv:2604.16548*. <https://arxiv.org/abs/2604.16548>

Louck, Y. (2026). Securing LLM-Agent Long-Term Memory Against Poisoning: Non-Malleable, Origin-Bound Authority with Machine-Checked Guarantees. *arXiv:2606.24322*. <https://arxiv.org/abs/2606.24322>

Ma, E., Zhou, Y., Huang, W.-C., Yang, J., Ma, H., Wang, Z., Li, C., Miao, C., Yu, P. S., and Wang, Z. (2026a). MEMPROBE: Probing Long-Term Agent Memory via Hidden User-State Recovery. *arXiv:2606.24595*. <https://arxiv.org/abs/2606.24595>

Maharana, A., Lee, D.-H., Tulyakov, S., Bansal, M., Barbieri, F., and Fang, Y. (2024). Evaluating Very Long-Term Conversational Memory of LLM Agents. *arXiv:2402.17753*. <https://arxiv.org/abs/2402.17753>

Ouyang, C., and Hou, R. (2026). MemLineage: Lineage-Guided Enforcement for LLM Agent Memory. *arXiv:2605.14421*. <https://arxiv.org/abs/2605.14421>

Packer, C., Wooders, S., Lin, K., Fang, V., Patil, S. G., Stoica, I., and Gonzalez, J. E. (2023/2024). MemGPT: Towards LLMs as Operating Systems. *arXiv:2310.08560*. <https://arxiv.org/abs/2310.08560>

Park, J. S., O'Brien, J. C., Cai, C. J., Morris, M. R., Liang, P., and Bernstein, M. S. (2023). Generative Agents: Interactive Simulacra of Human Behavior. *arXiv:2304.03442*. <https://arxiv.org/abs/2304.03442>

Pulipaka, S., Hlebig, S., Raghav, L., Abdelnabi, S., Raina, V., Sheth, I., and Fritz, M. (2026). Hidden in Memory: Sleeper Memory Poisoning in LLM Agents. *arXiv:2605.15338*. <https://arxiv.org/abs/2605.15338>

Sharma, T. (2026). SMSR: Certified Defence Against Runtime Memory Poisoning in Persistent LLM Agent Systems. *arXiv:2606.12703*. <https://arxiv.org/abs/2606.12703>

Shinn, N., Cassano, F., Berman, E., Gopinath, A., Narasimhan, K., and Yao, S. (2023). Reflexion: Language Agents with Verbal Reinforcement Learning. arXiv:2303.11366. <https://arxiv.org/abs/2303.11366>

Sunil, B. D., Sinha, I., Maheshwari, P., Todmal, S., Mallik, S., and Mishra, S. (2026). Memory Poisoning Attack and Defense on Memory Based LLM-Agents. arXiv:2601.05504. <https://arxiv.org/abs/2601.05504>

Tagliabue, V., and Dung, L. (2025/2026). Probing the Preferences of a Language Model: Integrating Verbal and Behavioral Tests of AI Welfare. arXiv:2509.07961. <https://arxiv.org/abs/2509.07961>

Wang, G., Xie, Y., Jiang, Y., Mandlekar, A., Xiao, C., Zhu, Y., Fan, L., and Anandkumar, A. (2023). Voyager: An Open-Ended Embodied Agent with Large Language Models. arXiv:2305.16291. <https://arxiv.org/abs/2305.16291>

Yang, T., Paul, S., Srinivasan, V., Kulkarni, V., and Chappidi, S. (2026). TRUSTMEM: Learning Trustworthy Memory Consolidation for LLM Agents with Long-Term Memory. arXiv:2606.25161. <https://arxiv.org/abs/2606.25161>

Yu, Y., Yao, L., Xie, Y., Tan, Q., Feng, J., Li, Y., and Wu, L. (2026). Agentic Memory: Learning Unified Long-Term and Short-Term Memory Management for Large Language Model Agents. arXiv:2601.01885. <https://arxiv.org/abs/2601.01885>

Zhang, X., Zheng, Y., Xu, Z., Zhou, K., Shen, B., Ou, H., Zhang, T., and Lam, K.-Y. (2026). MemMorph: Tool Hijacking in LLM Agents via Memory Poisoning. arXiv:2605.26154. <https://arxiv.org/abs/2605.26154>

Zhang, H., Zhang, S., Li, C., Wu, S., Shi, C., Zhu, X., and Zhang, H. (2026). Self-Harness: A Self-Improving Framework for Autonomous LLM Agents. arXiv:2606.09498. <https://arxiv.org/abs/2606.09498>

Zhang, Y. (2026). Agent libOS: A Library-OS-Inspired Runtime for LLM Agents. arXiv:2606.03895. <https://arxiv.org/abs/2606.03895>

Zhou, W., Zhou, X., Han, S., Xu, H., Li, G., Li, Z., Xiong, F., and Wu, F. (2026a). Are We Ready For An Agent-Native Memory System? arXiv:2606.24775. <https://arxiv.org/abs/2606.24775>

Zhou, C., Chai, H., Chen, W., Guo, Z., Shan, R., Song, Y., Xu, T., Yang, Y., Yu, A., Zhang, W., Zheng, C., Zhu, J., Zheng, Z., Zhang, Z., Lou, X., Zhang, C., Fu, Z., Wang, J., Liu, W., Lin, J., and Zhang, W. (2026b). Externalization in LLM Agents: A Unified Review of Memory, Skills, Protocols and Harness Engineering. arXiv:2604.08224. <https://arxiv.org/abs/2604.08224>

Zou, W., Dong, M., Calvo, M. R., Chang, S., Guo, J., Lee, D., Niu, X., Ma, X., Qi, Y., and Jiang, J. (2026). Poison Once, Exploit Forever: Environment-Injected Memory Poisoning Attacks on Web Agents. arXiv:2604.02623. <https://arxiv.org/abs/2604.02623>